creationDate	modifiedDate	tags	p
2026-07-11 12:45	2026-07-23 15:25	architecture, book-notes, programming	[[Boc

The Pragmatic Programmer - 20th Anniversary Edition

Note: This isn't as detailed as my usual notes, since I listed to the audiobook while working out and just jotted down things that stood out.

- Communication is an important skill.
- ETC ('Easier to Change') is the underlying rationale behind most of the book's advice, including many design patterns. Always keep ETC in mind when making design decisions.
- DRY, “Don’t repeat yourself.” Every piece of knowledge should have a single, unambiguous, authoritative representation within a system. DRY is a good example of ETC. If you repeat yourself when things change they have to be updated in multiple places. Remember, programming always includes maintenance. Even before you ship, requirements change daily.
- You can’t plan for things to remain the same. Change is constant.
- Make things orthogonal and easy to change since a change doesn’t ripple. Hide details behind an abstraction layer so when they change only the abstraction layer needs updating.
- Use “tracer bullet” code to get immediate feedback on if you’re hitting you goals. The difference between tracer bullets and a prototype is tracer bullets are well designed, production grade code that provide feedback and go on to form the skeleton of your application. Prototypes are designed to be thrown away. The value of a prototype is in the lessons learned.
- Estimate to avoid surprises. Consider estimating a likely, best case, and worst case scenario rather than sticking with one number and choose a unit that signifies your confidence level (days for less than 2 weeks, weeks for 3-6 weeks, months for 8-20 weeks, and don’t estimate anything over 20 weeks). You should typically spend more time on getting right parameters that are multiplied in than added, since they usually carry more weight on the final outcome. Record your estimates and determine why they differed from the final outcome when they do.

- Design by Contract in a nutshell: If you meet the preconditions of a function, it will make sure the post conditions and any invariants are true at the end of execution. If the contract is violated (perhaps by passing a negative value if one of the preconditions is positive values), it will throw an exception or terminate execution.
- The Law of Demeter simplified. Don't chain method calls. Object A shouldn't be that aware of the internal structure of Object B. If you're meant to access something through Object B, there should be a top level method on it that knows how to get to those internal values. Keep your code "shy;" it should only deal with things it knows about directly.
- FSM's (finite state machines) have all sorts of uses beyond computational theory. They can be represented as simple as a map of states to (events to new states). Always know what you are going to do in response to particular events even if it is as simple as 'error.' A more complicated example may associate an event in a given state with a transition function and a new state.
- PubSub is a good example of decoupling the handling of asynchronous events. Publisher and subscribers are connected via channels, subscribers register interest in channels, and publishers publish events to those channels. The downside is that it can be hard to see at a glance everything that happens in response to an event, since you don't have a central list of affected subscribers. Streams are similar to PubSub but model events as a live collection, so you can apply normal collection operations (map, filter, fold) directly to a sequence of events over time.
- All code translates inputs into outputs. It can be useful to think of your program as a data transformation pipeline. This naturally produces loosely coupled code. You can wrap the data in an Option type so an error doesn't disrupt the chain. The Option lets you chain transformations without null checking every step.
- Favor interfaces & protocols, delegation, and mixing & traits over inheritance. Inheritance couples your code to an entire tree of objects.
- Hide config behind an API layer to abstract away implementation details from the rest of the app.
- Make sure config can be updated without restarting the app. A PubSub mechanism can notify components of config changes, but use an existing library rather than rolling your own.
- Concurrency with shared mutable state is hard, even in functional languages, since real-world state still has to be dealt with somewhere. Consider alternatives to shared state:
    - Actors: each actor owns its internal state and processes one message at a time, to completion. No implicit concurrency is needed because there's no shared state to protect.

    - Blackboard architecture: a shared space (often a messaging system in microservices) that multiple components read from and write to. Because interactions are indirect, this is harder to reason about and requires a central repository of message formats and APIs. A unique trace ID per request helps you follow a process across the logs.
- Don’t program by coincidence. If you don’t know why something works, you won’t understand why it fails.
- Remember: version control, ruthless testing, full automation.